

Fachinformatiker/in

Lernfeld 7 – Cyber-physische Systeme ergänzen

LS: Durchführung eines Projektes mit Hilfe des Arduinos

Projekthandout

Arduino GameBoy – Handheld-Spielekonsole

Teilnehmer: [Namen der Teilnehmer]

Datum: 29. Mai 2026

Klasse: [Klasse eintragen]

Betreuer: [Name der Lehrkraft]

1 Formulierung des Themas

1.1 Thema

Entwicklung einer Handheld-Spielekonsole auf Basis des Arduino Mega 2560 – nachfolgend *Arduino GameBoy* genannt. Das Gerät vereint mehrere Spiele in einem kompakten, tragbaren Gehäuse und wird über ein OLED-Display, einen analogen Joystick sowie zwei Aktionstasten bedient.

1.2 Projektidee und Motivation

Ziel des Projektes ist die Konzeption und Umsetzung eines cyber-physischen Systems, das die im Unterricht erarbeiteten Grundlagen – Ansteuerung von Displays, Verwendung von Tastern, Einsatz von Interrupts und Auslesen analoger Sensoren – in einer praktischen Anwendung bündelt. Die Idee einer Spielekonsole wurde gewählt, weil sie einerseits einen hohen motivierenden Charakter besitzt und andererseits eine Vielzahl technischer Herausforderungen bietet: Echtzeitsteuerung, Speicherverwaltung, Zustandsautomaten und eine modulare Softwarearchitektur.

1.3 Projektziele

Die fertige Konsole soll folgende Anforderungen erfüllen:

- **Homescreen** mit horizontaler Spielauswahl über animierte Karten
- **Mehrere Spiele:** Snake („Origin“), Space Invaders („Infinity“), Dino Jump („Phallus“), Vampire Survivors („Godly“) und ein Raketen-Höhenspiel („Eternal“)
- **Pause-Funktion** in jedem Spiel mit Optionen für Weiter, Neustart und Zurück zum Menü
- **Highscore-Speicherung** im EEPROM – auch nach Stromverlust erhalten
- **Soundausgabe** über einen aktiven Buzzer für akustisches Feedback
- **Erweiterbare Architektur:** Neue Spiele lassen sich mit geringem Aufwand über eine einheitliche Schnittstelle hinzufügen
- **Stabiles 30 FPS Gameplay** trotz begrenzter Rechenleistung des Mikrocontrollers

2 Ressourcen und Ablaufplanung

2.1 Sachmittel und Kostenplanung

2.2 Zeitliche Ablaufplanung

Der Bearbeitungszeitraum erstreckte sich vom **[Startdatum]** bis zum 29.05.2026. Die folgende Tabelle gibt einen Überblick über die wesentlichen Projektphasen.

Komponente	Anzahl	Kosten (ca.)
Arduino Mega 2560 (Rev. 3)	1	[EUR eintragen]
2.42" OLED-Display (SSD1309, SPI)	1	[EUR eintragen]
Joystick-Modul (VRX, VRY, SW)	1	[EUR eintragen]
Taster (Pull-Down)	2	[EUR eintragen]
Aktiver Buzzer	1	[EUR eintragen]
Widerstände, Jumper-Kabel, Steckbrett	–	[EUR eintragen]
LEGO-Steine (Gehäuse)	–	[EUR eintragen]
Gesamtkosten:		[EUR eintragen]

Tabelle 1: Hardware-Komponenten und Kosten

Phase	Tätigkeit	Zeitraum
1	Konzeption: Anforderungen definieren, Hardware auswählen, Spieleideen skizzieren	[KW X]
2	Basisaufbau: Display in Betrieb nehmen, Joystick und Taster anbinden, Software-Grundgerüst	[KW X]
3	Spieleentwicklung: Homescreen, Space Invaders, Dino Jump	[KW X]
4	Erweiterung: Vampire Survivors, Snake, Eternal	[KW X]
5	Gehäusebau, Feinabstimmung, Pause-Logic, Highscore-EEPROM	[KW X]
6	Dokumentation, Präsentation, Videoerstellung	[KW X]

3 Durchführung

3.1 Vorgehensweise und Prozessschritte

Die Entwicklung folgte einem iterativen Ansatz. Zunächst wurde die grundlegende Hardware-Umgebung aufgebaut und getestet (Display-Ansteuerung, Eingabe über Joystick/Taster). Anschließend entstand eine modulare Software-Architektur mit einer abstrakten **Game-Basisklasse**, die es erlaubt, Spiele unabhängig voneinander zu entwickeln und über einen zentralen Zustandsautomaten zu verwalten.

1. **Hardware-Aufbau:** Verkabelung aller Komponenten gemäß Schaltplan (s. Anlage). Das OLED-Display kommuniziert per Hardware-SPI (MOSI=51, SCK=52), der Joystick liefert analoge X/Y-Werte über A0/A1, der Joystick-Taster hängt an Pin 2 (INPUT_PULLUP). Zwei externe Taster mit Pull-Down-Widerständen an Pin 30 und 32 dienen als Aktions- bzw. Pause-Taste. Ein aktiver Buzzer an Pin 40 erzeugt akustische Rückmeldungen.
2. **Software-Architektur:** Trennung in Hardware-Abstraktion (`hardware.h/.cpp`), Spiele-Basisklasse (`game_base.h`), Sound-Manager (`sound.h/.cpp`), Homescreen (`homescreen.h/.cpp`) sowie je eine Header/Source-Kombination pro Spiel. Das Hauptprogramm (`GameBoy.ino`) implementiert einen Zwei-Zustands-Automaten (Homescreen ↔ Spiel).
3. **Eingabesystem:** Implementierung einer entprellten Button-Klasse mit Flankenerkennung (`consumePress()`). Zusätzlich werden Pin-Change-Interrupts (PCINT2 für PORTC, INT4 für Joystick-SW) genutzt, um Tastendrucke auch während laufender Spiel-Logik zuverlässig zu erfassen.
4. **Display-Rendering:** Verwendung der U8g2-Bibliothek im Full-Framebuffer-Modus. Alle Spiele-Grafiken werden mit schnellen Zeichenprimitiven (`drawBox()`, `drawDisc()`),

`drawXBMP()`) umgesetzt, verzögerungsanfällige Operationen wie `drawLine()` wurden vermieden.

5. **Spieleentwicklung:** Jedes Spiel implementiert eine eigene Zustandsmaschine (Menü → Spielend → Pausiert → Game-Over) und trennt Zeichenmethoden strikt in `drawGame()` (ohne `sendBuffer()`) und `drawPlaying()` / `drawPause()` (mit `sendBuffer()`), um Flackern zu vermeiden.

3.2 Schaltplan / Visualisierung

Ein detaillierter Schaltplan mit allen Pin-Verbindungen befindet sich in der Anlage. Die zentrale Pin-Belegung ist:

Komponente	Pin	Typ	Hinweis
OLED CS	10	Digital	SPI Chip-Select
OLED DC	9	Digital	SPI Data/Command
OLED RST	8	Digital	Display-Reset
OLED MOSI	51	HW-SPI	Hardware SPI
OLED SCK	52	HW-SPI	Hardware SPI
Joystick X	A0	Analog	X-Achse (links/rechts)
Joystick Y	A1	Analog	Y-Achse (hoch/runter)
Joystick SW	2	Digital	Taster im Joystick
Button 1	32	Digital	Aktion (Schießen/Springen/Bestätigen)
Button 2	30	Digital	Pause / Zurück
Buzzer	40	Digital	Aktiv (HIGH = ein)

3.3 Aufgetretene Schwierigkeiten

Während der Entwicklung traten mehrere Herausforderungen auf:

- **Display zu klein:** Das ursprünglich vorgesehene Display erwies sich als zu klein für eine komfortable Spieleanzeige. Es wurde durch ein 2.42" OLED mit 128×64 Pixeln ersetzt.
- **Arbeitsspeicher unzureichend:** Der Arduino Uno verfügt nur über 2 KB SRAM – nicht genug für den Framebuffer (1 KB), mehrere Spielzustände und Objekt-Pools. Die Lösung war der Umstieg auf einen Arduino Mega 2560 mit 8 KB SRAM.
- **Gehäuse-Integration:** Alle Komponenten mit korrekten Pin-Verbindungen in ein LEGO-Gehäuse zu integrieren, erforderte mehrere Anläufe, bis alle Kabel sauber verlegt und die Taster mechanisch stabil montiert waren.
- **Pause-Funktion:** Das Unterbrechen des laufenden Spielgeschehens und das korrekte Wiederaufnehmen erwies sich als komplex. Die Lösung bestand im Einsatz von Pin-Change-Interrupts, die einen Pause-Flag setzen, der im Hauptloop ausgewertet wird. So wird die Spiel-Logik zuverlässig angehalten, während der Pause-Bildschirm weiterhin Eingaben verarbeiten kann.

4 Projektergebnis

4.1 Soll-Ist-Vergleich

Anforderung	Soll	Ist
Homescreen	Spielauswahl mit Vorschau	Erfüllt: Horizontales Karten-Layout mit weicher Scroll-Animation, 16×16 Icons, Spielname unter jedem Bild
Spieleanzahl	Mind. 2 Spiele	Übertroffen: 5 Spiele implementiert (Snake, Space Invaders, Dino Jump, Vampire Survivors, Rocket-Launch)
Highscore	Punktezähler während des Spiels	Erfüllt: Persistente Speicherung im EEPROM pro Spiel, Anzeige im Menü und Game-Over-Bildschirm
Pause / Menü	Pausieren, Fortsetzen, Neustart	Erfüllt: Debounced Menu-Navigation, 300 ms Guard gegen versehentliches Doppel-Pausieren
Sound	Akustisches Feedback	Erfüllt: Nicht-blockierender Buzzer mit SoundManager
Performance	Flüssiges Gameplay	Erfüllt: Stabile 30 FPS durch Frame-Timer (33 ms Intervall), optimierte Zeichenprimitive
Erweiterbarkeit	Neue Spiele einfach hinzufügbare	Erfüllt: Einheitliche Game-Schnittstelle, neues Spiel in 8 Schritten integrierbar
Gehäuse	Tragbares Gerät	Erfüllt: Kompaktes LEGO-Gehäuse mit allen Komponenten

4.2 Qualitätskontrolle

Die Konsole wurde in mehreren Test-Sessions auf Herz und Nieren geprüft: sämtliche Spiele wurden auf korrektes Verhalten bei Game-Over, Pause und Menü-Rückkehr getestet. Die Highscore-Persistenz wurde durch Stromtrennung verifiziert. Das Gehäuse hält auch intensiverem Tastendruck stand, und die Steckverbindungen bleiben auch bei Bewegung des Gerätes stabil.

4.3 Abweichungen und Anpassungen

Die wesentlichen Abweichungen vom ursprünglichen Plan – Umstieg auf ein größeres Display und den leistungsfähigeren Mega 2560 – haben den Projektumfang eher erweitert als eingeschränkt. Der zusätzliche Speicher ermöglichte erst die Umsetzung komplexerer Spiele wie Vampire Survivors und Eternal, die umfangreiche Objekt-Pools und Zustandsautomaten benötigen.

4.4 Ausblick

Das Projekt bietet zahlreiche Anknüpfungspunkte für Weiterentwicklungen:

- **Weitere Spiele:** Die modulare Architektur erlaubt das einfache Hinzufügen von Tetris, Pong oder Jump-'n'-Run-Varianten.
- **Multiplayer:** Über die serielle Schnittstelle (UART) des Mega könnten zwei Geräte für kompetitive Spiele gekoppelt werden.
- **Akku-Betrieb:** Ein Li-Ion-Akku mit Ladeelektronik würde das Gerät vollständig mobil machen.
- **Gehäuse-Veredelung:** Ein 3D-gedrucktes, ergonomisch optimiertes Gehäuse böte mehr Komfort und eine professionellere Optik.
- **Ton-Verbesserung:** Ein passiver Lautsprecher mit PWM-Ansteuerung könnte Melodien statt einfacher Pieptöne erzeugen.

5 Anlage

Folgende Materialien sind diesem Handout beigelegt bzw. liegen im Moodle-Abgabeordner vor:

1. Schaltplan (Fritzing / handgezeichnet)
2. Vollständiger Programmcode (siehe Moodle-Abgabe)
3. Screenshots aller Spiele und des Homescreens
4. Video: Aufbau und Funktion der Konsole (siehe Moodle-Abgabe)
5. Präsentationsfolien (siehe Moodle-Abgabe)